使用BERT实现情感分析预训练模型进行情感分析过程

（以 bert-base-chinese 为例）

背景介绍

在 CV 问题中，目前已经有了很多成熟的模型供大家使用，只需要结合特定的业务场景修改结尾的全连接层或添加 softmax 层即可满足需求，也就是我们常说的迁移学习。那么在 NLP 领域是否有这样泛化能力很强的模型呢，答案是肯定的。

2018 年底，Google 推出了一个打破 11 项 NLP 任务的模型 BERT（Bidirectional Encoder Representation from Transformers），该模型一经问世就火遍 AI 领域并受到了广大开发者的青睐，可以说是 NLP 领域中具有里程碑意义的模型，目前 BERT 依旧是比赛中或者工业界首选的模型，各大公司也均基于 BERT 进行了更多的升级与优化。

BERT 是一个预训练模型，该模型的训练阶段分为两个部分，预训练与微调，预训练阶段 Google 已经处理好，如果要使用该模型，只需要针对特定场景进行微调即可。在本章节中，我们会先介绍 BERT 的原理，再以一个实际的例子来讲解。

前置知识

1. Transformer 模型

在 NLP 任务中，常用的特种提取器有 RNN 及其变体、CNN 搭配池化层、Transformer 等，RNN 类型的提取器有一个最大的优点能捕捉长依赖信息，但是其速度很慢，CNN 搭配池化层能有效获取一些重要的特征并忽略没有意义的特征，但是却无法捕捉长依赖信息，Transformer 兼具了 RNN 与 CNN 的优点，在保留长依赖信息的同时速度也很快，其中的 attention 机制也使其具有了类似最大池化层捕捉重要特征的能力。BERT 的特征提取器实际上就是采用的 Transformer 的 encoder 层，Google 提供了两个版本的 BERT，其中 base 版本的是由 12 层的 Transformer 的 encoder 堆叠在一起，large 版本的是由 24 层的 Transformer 的 encoder 堆叠在一起。

Transformer 模型是基于 encoder-decoder 结构的，如图 10.1-1 所示。其中 encoder 层由 6 层图中所示的结构堆叠而成，每一个层又包括了两个子层，第一个子层是 multi-head self-attention，第二个子层是一个全连接层，这两个子层均采用了 residual connection 来进行连接，并且还有一层 layer normalization 层。decoder 和 encoder 的结构类似，也是由 6 层图中所示的结构堆叠而成，除了 encoder 提到的两层结构外，decoder 层还有一层额外的 masked multi-head attention。

attention 是一种加权机制，针对候选值进行加权求和。Attention 能表述为一个 query 与一个

key-value 集合的映射关系，其中 query，keys，values 都是向量，输出值是 values 的加权求和，这个权重是根据 query 与 key 来计算出来的。

2. BERT 预训练

BERT 的 base 版本是由 12 个 transformer 的 encoder 层堆叠在一起，没有用到 decoder 层，因此 transformer 的 decoder 这里就不再赘述，感兴趣的读者可自己阅读 transformer 的论文。接下来我们就一起来看下 BERT 的训练阶段是怎么做的。

BERT 的预训练阶段采用了两个独有的非监督任务，一个是 Masked Language Model，还有一个是 Next Sentence Prediction。

3. 使用 BERT 进行文本分类

接下来我们以一个情感分析的例子来介绍如何在 keras 中使用对 bert 进行微调。情感分析的应用场景很多，也是 nlp 领域一个最常见的任务，本文将会以 IMDB 数据集为例，该数据集是一个开源的电影评价数据集，我们的任务是根据文本信息判断评价是正面情绪还是负面情绪。

案例：数据预处理

数据的读取、处理、再保存备用

1）通过 pd.read_csv 函数读取数据，该函数是用来读取 csv 格式的文件，将表格数据转化成 dataframe 格式。dataframe 格式的数据在操作时极为方便，由于我们仅需要评论和评分这两列，所以通过索引取出对应的数据。

[1]:
```
#导入数据处理的基础包
import numpy as np
import pandas as pd
#导入用于计数的包
from collections import Counter
import os
import requests
```

[2]:
```
#读取数据
#路径是各自数据路径的存放地址
data = pd.read_csv('JDcontents_vivo.csv',encoding="gbk")
#输出数据的一些相关信息
print(data.info())
#看数据形状（行数，列数）
print(data.shape)
# 列标签 <Index>
print(data.columns)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11996 entries, 0 to 11995
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   评论      11996 non-null  object
 1   评分      11996 non-null  int64 
dtypes: int64(1), object(1)
memory usage: 187.6+ KB
None
(11996, 2)
Index(['评论', '评分'], dtype='object')
```

[3]:
```
#观察数据格式，分别查看data的前五条数据和后五条数据
data.head()
data.tail()
```

[3]:

	评论	评分
11991	包装很漂亮，看起来高端大气上档次，比某米看起来档次高	5
11992	外形外观：不错 屏幕音效：不错拍照效果：还行运行速度：刚刚的	5
11993	手机不错，1199买的，618活动1299，619凌晨0点降价100，。。。然后保价，赠品变...	5
11994	是1049	5
11995	正品IQoo，手机颜色好看很好看，运行速度快，价格实惠。拍照清晰解锁快。价格实惠屏幕显示清晰...	5

2）查看数据后发现，数据的“评论”中存在重复现象，所以需要将“评论”一列进行去重处理。去除重复的评论后，由于重复评论的一整行都会被删除，所以会出现列序号索引混乱，需要对索引进行重新排序。

[4]:
```
#目的是将label的数值 -1 缩放到[0,4]之间
def get_label(label):
    label = label - 1
    return label
data["label"] = data['评分'].apply(get_label)
```

[5]:
```
#按评论进行去重，对于重复项，保留第一次出现的值
data = data.drop_duplicates('评论',keep='first')
#会将标签重新从零开始顺序排序,使用参数设置drop=True删除旧的索引序列
data = data.reset_index(drop=True)
```

[6]:
```
# TODO1: 去掉一些无用的字符，自行定一个字符几何，并从文本中去掉
#去除字母数字表情和其它字符
import re
def clear_character(sentence):
    pattern1='[a-zA-Z0-9]'
    pattern2 = re.compile(u'[^\s1234567890:: ' + '\u4e00-\u9fa5]+')
    pattern3='[’!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~]+'
    line1=re.sub(pattern1,'',sentence)  #去除英文字母和数字
    line2=re.sub(pattern2,'',line1)  #去除表情和其他字符
    line3=re.sub(pattern3,'',line2)  #去除去掉残留的冒号及其它符号
    new_sentence=''.join(line3.split()) #去除空白
    return new_sentence
```

[7]:
```
#在["评论"]这一列使用定义的"clear_character"函数
data["comment_processed"]=data['评论'].apply(clear_character)
data.head()
```

[7]:

	评论	评分	label	comment_processed
0	您没有填写内容，默认好评	1	0	您没有填写内容默认好评
1	充电慢，快递员也不好说话！	1	0	充电慢快递员也不好说话
2	价位比实体店还贵一百多	1	0	价位比实体店还贵一百多
3	说是送蓝牙耳机，结果也没送，问客服说送完了。	1	0	说是送蓝牙耳机结果也没送问客服说送完了
4	刚买就便宜120，又没价格保护	1	0	刚买就便宜又没价格保护

3）jieba 分词、去除停用词、去除停用词

[9]:
```
# 导入中文分词包jieba，并用jieba对原始文本做分词
import jieba
from tqdm import tqdm
def comment_cut(content):
    # TODO: 使用结巴完成对每一个comment的分词
#     seg = jieba.lcut(content)
    seg = list(jieba.cut(content.strip()))
    return seg
# 输出进度条
tqdm.pandas(desc='apply')
data['comment_processed'] = data['comment_processed'].progress_apply(comment_cut)
# 观察新的数据的格式
data.head()
```

```
apply:   0%|
| 0/7449 [00:00<?, ?it/s]Building prefix dict from the default dictionary ...
Dumping model to file cache C:\Users\12722\AppData\Local\Temp\jieba.cache
Loading model cost 0.759 seconds.
Prefix dict has been built successfully.
apply: 100%|
| 7449/7449 [00:02<00:00, 3163.26it/s]
```

[9]:

	评论	评分	label	comment_processed
0	您没有填写内容，默认好评	1	0	[您, 没有, 填写内容, 默认, 好评]
1	充电慢，快递员也不好说话！	1	0	[充电, 慢, 快递, 员, 也, 不好, 说话]
2	价位比实体店还贵一百多	1	0	[价位, 比, 实体店, 还贵, 一百多]
3	说是送蓝牙耳机，结果也没送，问客服说送完了。	1	0	[说, 是, 送, 蓝牙, 耳机, 结果, 也, 没, 送, 问, 客服, 说, 送, 完, 了]
4	刚买就便宜120，又没价格保护	1	0	[刚买, 就, 便宜, 又, 没, 价格, 保护]

[26]:
```
# 设定停用词并从文本中去掉停用词
# 下载中文停用词表至本地目录下/root/Jupyter/Data_cleaning/stopWord中，下载地址:https://github.com/goto456/stopwords/
if not os.path.exists('stopWord.json'):
    stopWordC=""
    with open("cn_stopwords.txt","r",encoding='utf-8') as f:
        stopWordC = f.read()
        #print(stopWord)
    #stopWord = requests.get("https://raw.githubusercontent.com/goto456/stopwords/master/cn_stopwords.txt")
    with open("stopWord.json", "wb") as f1:
         f1.write(stopWordC)
# 读取下载的停用词表，并保存在列表中
#with open("stopWord.json","r",encoding='utf-8') as f:
#    stopWords = f.read().split("\n")
with open("cn_stopwords.txt","r",encoding='utf-8') as f:
    stopWords = f.read().split("\n")

# 去除停用词
def rm_stop_word(wordList):
    # your code, remove stop words
    # TODO
    #outstr = ''
    # 去停用词
    #for word in wordList:
       # if word not in stopWords:
           # if word != '\t':
                #outstr += word
                #outstr += " "
    #return outstr
    filtered_words = [word for word in wordList if word not in stopWords]
    return filtered_words
    #return " ".join(filtered_words)
#这行代码中.progress_apply()函数的作用等同于.apply()函数的作用，只是写成.progress_apply()函数才能被tqdm包监控从而输出进度条。
data['comment_processed'] = data['comment_processed'].progress_apply(rm_stop_word)
# 观察新的数据的格式
data.head()
```

```
apply: 100%|██████████| 7449/7449 [00:02<00:00, 3466.41it/s]
```

[26]:

	评论	评分	label	comment_processed
0	您没有填写内容，默认好评	1	0	[没有, 填写内容, 默认, 好评]
1	充电慢，快递员也不好说话！	1	0	[充电, 慢, 快递, 员, 不好, 说话]
2	价位比实体店还贵一百多	1	0	[价位, 实体店, 还贵, 一百多]
3	说是送蓝牙耳机，结果也没送，问客服说送完了。	1	0	[说, 送, 蓝牙, 耳机, 没, 送, 问, 客服, 说, 送, 完]
4	刚买就便宜120，又没价格保护	1	0	[刚买, 便宜, 没, 价格, 保护]

[27]:
```
# 去除低频词，去掉词频小于10的单词，并把结果存放在data['comment_processed']里
from collections import Counter
list_set = []
for i in range(len(data)):
    for j in data.iloc[i]['comment_processed']:
        list_set.extend(j)

words_count = Counter(list_set)

min_threshold=10
my_dict = {k: v for k, v in words_count.items() if v < min_threshold}
filteredA = Counter(my_dict)

# 去除低频词
def rm_low_frequence_word(wordList):
    # your code, remove stop words
    # TODO
    outstr = ''
    for word in wordList:
        if word not in filteredA:
            if word != '\t':
                outstr += word
                outstr += " "
    #filtered_words = [word for word in wordList if word not in filteredA]
    return outstr

#这行代码中.progress_apply()函数的作用等同于.apply()函数的作用，只是写成.progress_apply()函数才能被tqdm包监控从而输出进度条。
data['comment_processed'] = data['comment_processed'].progress_apply(rm_low_frequence_word)
data.head()
```

```
apply: 100%|██████████| 7449/7449 [00:00<00:00, 158914.42it/s]
```


```
apply: 100%|██████████| 7449/7449 [00:00<00:00, 158914.42it/s]
```


[27]:

	评论	评分	label	comment_processed
0	您没有填写内容，默认好评	1	0	没有 填写内容 默认 好评
1	充电慢，快递员也不好说话！	1	0	充电 慢 快递 员 不好 说话
2	价位比实体店还贵一百多	1	0	价位 实体店 还贵 一百多
3	说是送蓝牙耳机，结果也没送，问客服说送完了。	1	0	说 送 蓝牙 耳机 没 送 问 客服 说 送 完
4	刚买就便宜120，又没价格保护	1	0	刚买 便宜 没 价格 保护

4）数据保存、与处理后数据的查看

[29]:
```
#index参数设置为False表示不保存行索引,header设置为False表示不保存列索引
data.to_csv("JDcontents_vivo_Cleaned_data.csv",index=False,header=True,encoding="utf-8")
```

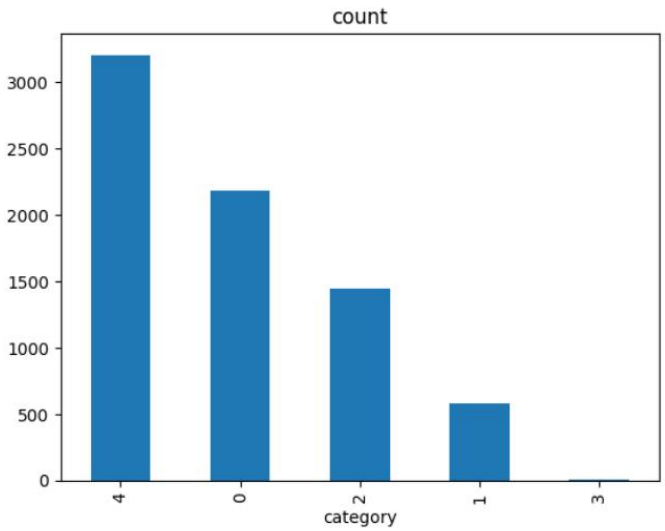
[30]:
```
#读取数据
#路径是各自数据路径的存放地址
data = pd.read_csv('JDcontents_vivo_Cleaned_data.csv')
#输出数据的一些相关信息
print(data.info())
#看数据形状（行数，列数）
print(data.shape)
# 列标签 <Index>
print(data.columns )
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7449 entries, 0 to 7448
Data columns (total 4 columns):
 #   Column             Non-Null Count  Dtype 
---  ------             --------------  ----- 
 0   评论                 7449 non-null   object
 1   评分                 7449 non-null   int64 
 2   label              7449 non-null   int64 
 3   comment_processed  7428 non-null   object
dtypes: int64(2), object(2)
memory usage: 232.9+ KB
None
(7449, 4)
Index(['评论', '评分', 'label', 'comment_processed'], dtype='object')
```

[29]:
```
#index参数设置为False表示不保存行索引,header设置为False表示不保存列索引
data.to_csv("JDcontents_vivo_Cleaned_data.csv",index=False,header=True,encoding="utf-8")
```

[30]:
```
#读取数据
#路径是各自数据路径的存放地址
data = pd.read_csv('JDcontents_vivo_Cleaned_data.csv')
#输出数据的一些相关信息
print(data.info())
#看数据形状（行数，列数）
print(data.shape)
# 列标签 <Index>
print(data.columns )
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7449 entries, 0 to 7448
Data columns (total 4 columns):
 #   Column             Non-Null Count  Dtype 
---  ------             --------------  ----- 
 0   评论                 7449 non-null   object
 1   评分                 7449 non-null   int64 
 2   label              7449 non-null   int64 
 3   comment_processed  7428 non-null   object
dtypes: int64(2), object(2)
memory usage: 232.9+ KB
None
(7449, 4)
Index(['评论', '评分', 'label', 'comment_processed'], dtype='object')
```



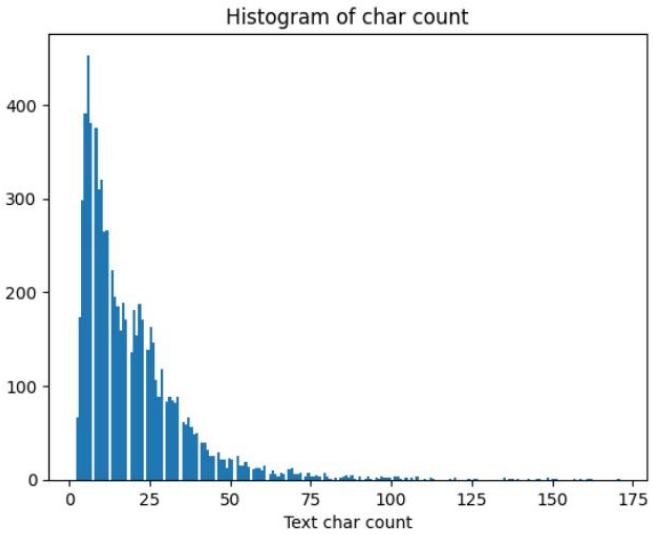
6]:
```
#统计单词的个数来得到每个句子的长度
data['text_len'] = data['comment_processed'].apply(lambda x: len(x.split(' ')))
print(data['text_len'].describe())
```

```
count    7428.000000
mean       19.494615
std        17.145183
min         2.000000
25%         8.000000
50%        14.000000
75%        26.000000
max       171.000000
Name: text_len, dtype: float64
```

[36]:
```
#统计单词的个数来得到每个句子的长度
data['text_len'] = data['comment_processed'].apply(lambda x: len(x.split(' ')))
print(data['text_len'].describe())
```

```
count    7428.000000
mean       19.494615
std        17.145183
min         2.000000
25%         8.000000
50%        14.000000
75%        26.000000
max       171.000000
Name: text_len, dtype: float64
```

[37]:
```
_ = plt.hist(data['text_len'], bins=200)
plt.xlabel('Text char count')
plt.title("Histogram of char count")
#plt.savefig('./text_chart_count.png')
plt.show()
```



案例：预训练模型进行情感分析数据模型处理生成过程

1）导入需要的库，并且定义数据路径，然后简单查看下数据

[1]:
```
import numpy as np
import pandas as pd
import csv
import torch.nn as nn
import torch
import transformers
from torch.optim.lr_scheduler import ReduceLROnPlateau
from torch.utils.data import TensorDataset, DataLoader
from transformers import BertTokenizer,BertConfig,AdamW,BertModel
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from tqdm import tqdm
```

[2]:
```
#读取数据
Data_path = "./JDcontents_vivo_Cleaned_data.csv"
Totle_data = pd.read_csv(Data_path)

train_dataset,temp_data =  train_test_split(Totle_data,test_size=0.2)
validate_dataset, test_dataset = train_test_split(temp_data,test_size=0.5)
print(len(train_dataset))
print(len(validate_dataset))
print(len(test_dataset))

#设置保存路径
train_data_path="./Train.csv"
dev_data_path = "./Dev.csv"
test_data_path="./Test.csv"

#index参数设置为False表示不保存行索引,header设置为False表示不保存列索引
train_dataset.to_csv(train_data_path,index=False,header=True)
validate_dataset.to_csv(dev_data_path ,index=False,header=True)
test_dataset.to_csv(test_data_path,index=False,header=True)
```

```
5942
743
743
```

[3]:
```
data = pd.read_csv(train_data_path)
data
```

[3]:

	label	comment_processed
0	2	耗油 厉害 刚买 几天 降价 保价
1	1	起来 不错 手感 有点 滑 容易 掉 问题 一个 非常 差 音质 苹果 接听 驱动 基本上 ...
2	4	家里 老人 买 手机 综合 评价 一下 外观 不错 颜色 鲜亮 炫彩 手感 舒服 外表 看起...
3	0	差 差
4	1	充电 慢死 慢死 一个 闪充 没 一个 普通 充电 手机 充 快 一个 小时 充 百分之六十
...	...	...
5937	1	手机 太 垃圾 刷 死机 玩游戏 死机 评论 京东 买 手机 六七个 垃圾
5938	4	新手机 超赞 包装 精美 颜色 漂亮 大屏幕 开机 试下 速度 快 操作 流畅 拍照 视频 ...
5939	2	天 还过得去 电池 真心 觉得 不好 耐用 知道 是不是 原装 电池
5940	0	非常 满意 买 领了 优惠券 下来 后来 一看 价格 已经 生气
5941	2	掉电 快 充电 快 屏幕 灵敏度

5942 rows × 2 columns

2）定义神经网络，使用 BertTokenizer 编码成 Bert 需要的输入格式，数据送入预训练模型之间需要进行预处理，使用 BertTokenizer 将数据编码为 Bert 需要的输入格式。预训练模型有三种输入分别是 input_ids、token_type_ids 、attention_mask。最后，将数据加载为 Tensor 格式

[4]:
```
class BertClassificationModel(nn.Module):
    def __init__(self):
        super(BertClassificationModel, self).__init__()
        #加载预训练模型
        pretrained_weights="Bert/bert-base-chinese/"
        self.bert = transformers.BertModel.from_pretrained(pretrained_weights)
        for param in self.bert.parameters():
            param.requires_grad = True
        #定义线性函数
        self.dense = nn.Linear(768, 2)  #bert默认的隐藏单元数是768， 输出单元是2，表示二分类

    def forward(self, input_ids,token_type_ids,attention_mask):
        #得到bert_output
        bert_output = self.bert(input_ids=input_ids,token_type_ids=token_type_ids, attention_mask=attention_mask)
        #获得预训练模型的输出
        bert_cls_hidden_state = bert_output[1]
        #将768维的向量输入到线性层映射为二维向量
        linear_output = self.dense(bert_cls_hidden_state)
        return  linear_output
```

[5]:
```
def encoder(max_len,vocab_path,text_list):
    #将text_list embedding成bert模型可用的输入形式
    #加载分词模型
    bert = BertModel.from_pretrained("Bert/bert-base-chinese/")
    tokenizer = BertTokenizer.from_pretrained(vocab_path)

    tokenizer = tokenizer(
        text_list,
        padding = True,
        truncation = True,
        max_length = max_len,
        return_tensors='pt'  # 返回的类型为pytorch tensor
        )
    input_ids = tokenizer['input_ids']
    token_type_ids = tokenizer['token_type_ids']
    attention_mask = tokenizer['attention_mask']
    return input_ids,token_type_ids,attention_mask
```

[6]:
```
def load_data(path):
    csvFileObj = open(path,encoding='UTF-8')
    readerObj = csv.reader(csvFileObj)
    text_list = []
    labels = []
    for row in readerObj:
        #跳过表头
        if readerObj.line_num == 1:
            continue
        #Label在什么位置就改成对应的index
        label = int(row[0])
        text = row[1]
        text_list.append(text)
        labels.append(label)
    #调用encoder函数，获得预训练模型的三种输入形式
    input_ids,token_type_ids,attention_mask = encoder(max_len=150,vocab_path="Bert/vocab.txt",text_list=text_list)
    labels = torch.tensor(labels)
    #将encoder的返回值以及label封装为Tensor的形式
    data = TensorDataset(input_ids,token_type_ids,attention_mask,labels)
    return data
```

3）实例化 DataLoader、验证函数的定义，训练函数的定义、实例化模型并进行训练与验证（很遗憾但是训练时报错，暂时没有解决，最后没有能生成训练模型）

[7]:
```
#设定batch_size
batch_size = 16
#引入数据路径
train_data_path="Train.csv"
dev_data_path="Dev.csv"
test_data_path="Test.csv"
#调用load_data函数，将数据加载为Tensor形式
train_data = load_data(train_data_path)
dev_data = load_data(dev_data_path)
test_data = load_data(test_data_path)
#将训练数据和测试数据进行DataLoader实例化
train_loader = DataLoader(dataset=train_data, batch_size=batch_size, shuffle=True)
dev_loader = DataLoader(dataset=dev_data, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(dataset=test_data, batch_size=batch_size, shuffle=False)
```

[8]:
```
def dev(model,dev_loader):
    #将模型放到服务器上
    model.to(device)
#设定模式为验证模式
    model.eval()
#设定不会有梯度的改变仅作验证
    with torch.no_grad():
        correct = 0
        total = 0
        for step, (input_ids,token_type_ids,attention_mask,labels) in tqdm(enumerate(dev_loader),desc='Dev Itreation:'):
            input_ids,token_type_ids,attention_mask,labels=input_ids.to(device),token_type_ids.to(device),attention_mask.to(device),labels.to(device)
            out_put = model(input_ids,token_type_ids,attention_mask)
            _, predict = torch.max(out_put.data, 1)
            correct += (predict==labels).sum().item()
            total += labels.size(0)
        res = correct / total
        return res
```

[9]:
```python
def train(model,train_loader,dev_loader) :
    #将model放到服务器上
    model.to(device)
    #设定模型的模式为训练模式
    model.train()
    #定义模型的损失函数
    criterion = nn.CrossEntropyLoss()
    param_optimizer = list(model.named_parameters())
    no_decay = ['bias', 'LayerNorm.bias', 'LayerNorm.weight']
    #设置模型参数的权重衰减
    optimizer_grouped_parameters = [
        {'params': [p for n, p in param_optimizer if not any(nd in n for nd in no_decay)],
         'weight_decay': 0.01},
        {'params': [p for n, p in param_optimizer if any(nd in n for nd in no_decay)], 'weight_decay': 0.0}
    ]
    #学习率的设置
    optimizer_params = {'lr': 1e-5, 'eps': 1e-6, 'correct_bias': False}
    #使用AdamW 主流优化器
    optimizer = AdamW(optimizer_grouped_parameters, **optimizer_params)
    #学习率调整器，检测准确率的状态，然后衰减学习率
    scheduler = ReduceLROnPlateau(optimizer,mode='max',factor=0.5,min_lr=1e-7, patience=5,verbose= True, threshold=0.0001, eps=1e-08)
    t_total = len(train_loader)
    #设定训练轮次
    total_epochs = 2
    bestAcc = 0
    correct = 0
    total = 0
    print('Training and verification begin!')
    for epoch in range(total_epochs):
        for step, (input_ids,token_type_ids,attention_mask,labels) in enumerate(train_loader):
#从实例化的DataLoader中取出数据，并通过 .to(device)将数据部署到服务器上     input_ids,token_type_ids,attention_mask,labels=input_ids.to(device),token_type_ids.
            #梯度清零
            optimizer.zero_grad()
            #将数据输入到模型中获得输出
            out_put =  model(input_ids,token_type_ids,attention_mask)
            #计算损失
            loss = criterion(out_put, labels)
            _, predict = torch.max(out_put.data, 1)
            correct += (predict == labels).sum().item()
            total += labels.size(0)
            loss.backward()
            optimizer.step()
             #每两步进行一次打印
            if (step + 1) % 2 == 0:
                train_acc = correct / total
                print("Train Epoch[{}/{}],step[{}/{}],tra_acc{:.6f} %,loss:{:.6f}".format(epoch + 1, total_epochs, step + 1, len(train_loader),train_acc*100,l
            #每五十次进行一次验证
            if (step + 1) % 50 == 0:
                train_acc = correct / total
                #调用验证函数dev对模型进行验证，并将有效果提升的模型进行保存
                acc = dev(model, dev_loader)
                if bestAcc < acc:
                    bestAcc = acc
                    #模型保存路径
                    path = 'savedmodel/span_bert_hide_model1.pkl'
                    torch.save(model, path)
                print("DEV Epoch[{}/{}],step[{}/{}],tra_acc{:.6f} %,bestAcc{:.6f}%,dev_acc{:.6f} %,loss:{:.6f}".format(epoch + 1, total_epochs, step + 1, len(
        scheduler.step(bestAcc)
```

[10]:
```python
device = torch.device('cuda:3' if torch.cuda.is_available() else 'cpu')
#实例化模型
model = BertClassificationModel()
#调用训练函数进行训练与验证
train(model,train_loader,dev_loader)
```

```
D:\MySoftware\Anaconda3\envs\StudyHuggingFaceBertEnv\lib\site-packages\transformers\optimization.py:411: FutureWarning: This implementation of AdamW is deprecated and will be removed in a future version. Use the PyTorch implementation torch.optim.AdamW instead, or set `no_deprecation_warning=True` to disable this warning
  warnings.warn(
Training and verification begin!
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[10], line 5
      3 model = BertClassificationModel()
      4 #调用训练函数进行训练与验证
----> 5 train(model,train_loader,dev_loader)

Cell In[9], line 37, in train(model, train_loader, dev_loader)
     35 out_put =  model(input_ids,token_type_ids,attention_mask)
     36 #计算损失
---> 37 loss = criterion(out_put, labels)
     38 _, predict = torch.max(out_put.data, 1)
     39 correct += (predict == labels).sum().item()

File D:\MySoftware\Anaconda3\envs\StudyHuggingFaceBertEnv\lib\site-packages\torch\nn\modules\module.py:889, in Module._call_impl(self, *input, **kwargs)
    887     result = self._slow_forward(*input, **kwargs)
    888 else:
--> 889     result = self.forward(*input, **kwargs)
    890 for hook in itertools.chain(
    891         _global_forward_hooks.values(),
    892         self._forward_hooks.values()):
    893     hook_result = hook(self, input, result)

File D:\MySoftware\Anaconda3\envs\StudyHuggingFaceBertEnv\lib\site-packages\torch\nn\modules\loss.py:1047, in CrossEntropyLoss.forward(self, input, target)
   1045 def forward(self, input: Tensor, target: Tensor) -> Tensor:
   1046     assert self.weight is None or isinstance(self.weight, Tensor)
-> 1047     return F.cross_entropy(input, target, weight=self.weight,
   1048                            ignore_index=self.ignore_index, reduction=self.reduction)
```

```
File D:\MySoftware\Anaconda3\envs\StudyHuggingFaceBertEnv\lib\site-packages\torch\nn\functional.py:2690, in cross_entropy(input, target, weight, size_average, ignore_index, reduce, reduction)
   2688 if size_average is not None or reduce is not None:
   2689     reduction = _Reduction.legacy_get_string(size_average, reduce)
-> 2690 return nll_loss(log_softmax(input, 1), target, weight, None, ignore_index, None, reduction)

File D:\MySoftware\Anaconda3\envs\StudyHuggingFaceBertEnv\lib\site-packages\torch\nn\functional.py:2385, in nll_loss(input, target, weight, size_average, ignore_index, reduce, reduction)
   2381     raise ValueError(
   2382         "Expected input batch_size ({}) to match target batch_size ({}).".format(input.size(0), target.size(0))
   2383     )
   2384 if dim == 2:
-> 2385     ret = torch._C._nn.nll_loss(input, target, weight, _Reduction.get_enum(reduction), ignore_index)
   2386 elif dim == 4:
   2387     ret = torch._C._nn.nll_loss2d(input, target, weight, _Reduction.get_enum(reduction), ignore_index)

IndexError: Target 4 is out of bounds.
```

4）定义预测函数、使用训练好的模型进行预测、获得预测值与预测的概率、调用函数计算准确率等指标

[11]:
```python
def predict(model,test_loader):
    model.to(device)
    model.eval()
    predicts = []
    predict_probs = []
    with torch.no_grad():
        correct = 0
        total = 0
        for step, (input_ids,token_type_ids,attention_mask,labels) in enumerate(test_loader):
            input_ids,token_type_ids,attention_mask,labels=input_ids.to(device),token_type_ids.to(device),attention_mask.to(device),labels.to(device)
            out_put = model(input_ids,token_type_ids,attention_mask)

            _, predict = torch.max(out_put.data, 1)

            pre_numpy = predict.cpu().numpy().tolist()
            predicts.extend(pre_numpy)
            probs = F.softmax(out_put).detach().cpu().numpy().tolist()
            predict_probs.extend(probs)

            correct += (predict==labels).sum().item()
            total += labels.size(0)
        res = correct / total
        print('predict_Accuracy : {} %'.format(100 * res))
        #返回预测结果和预测的概率
        return predicts,predict_probs
```

[ ]:
```python
#引进训练好的模型进行测试
path = 'savedmodel/span_bert_hide_model.pkl'
Trained_model = torch.load(path)
#predicts是预测的（0   1），predict_probs是概率值
predicts,predict_probs = predict(Trained_model,dev_loader)
```

[14]:
```python
P = sklearn.metrics.precision_score(y_true, y_pred, average='binary',sample_weight=None)
R = sklearn.metrics.recall_score(y_true, y_pred, average='binary',sample_weight=None)
F1 = sklearn.metrics.f1_score(y_true, y_pred,average='binary',sample_weight=None)
```

参数名	含义	类型
y_true	正确值	1维矩阵
y_pred	预测值	1维矩阵
average	计算类型	字符串，'binary'(默认)、'micro'、'macro'、'weighted'、'samples'
sample_weight	样本比重	n维矩阵（n=样本类数）

average的选项详解：

选项	含义
binary	二分类
micro	统计全局TP和FP来计算
macro	计算每个标签的未加权均值（不考虑不平衡）
weighted	计算每个标签等等加权均值（考虑不平衡）
samples	计算每个实例找出其均值

感谢结语

最后，感谢老师同学的观看。

本节使用的是京东对 vivo 的一些评论的情感分析例子，当然掌握的客户的情感分析的步骤，可

以用在不同场景进行应用，合理处理模型的分析的参数调整，改善模型分析的准确性需要不断的测试调整。

代码完善整理后，后面会一起传上去。